



SSE(Server-Sent Events) 선택 배경 및 의사결정 과정

1. 처음엔 WebSocket으로 시작했다
2. “그냥 polling이면 되지 않을까?”라는 생각
3. 기획 변경 및 프론트 피드백 → polling 한계 부각
4. “메인화면 진입 시 상태 조회” 방식 고려
5. 실시간 반응을 위한 대안 고민: WebSocket vs SSE
6. 최종 선택: SSE

1. 처음엔 WebSocket으로 시작했다

- 초기에는 사용자 요청에 대해 서버에서 비동기적으로 처리되는 작업의 완료 상태를 실시간으로 전달해야 한다는 요구가 있었음
- 특히 음악 생성 시간이 수 초 이상 소요되기 때문에, 사용자가 업로드 후 기다리는 동안 실시간 상태 업데이트가 필요하다고 판단
- WebSocket은 브라우저에서 폭넓게 지원되며, 실시간 양방향 통신에 널리 사용되는 기술이므로, 가장 직관적인 선택지로 보였음
- 프론트엔드에서도 작업 완료 시 즉시 결과를 받고 싶다는 요구가 있었고, 이를 구현하기 위해 WebSocket을 채택

BUT, WebSocket은 양방향, 지속 연결, 다양한 메시지 주고받기가 필요한 복합적인 상호작용에 적합함
→ 우리 서비스는 단순히 작업 완료 여부를 한 번 클라이언트에 알려주기만 하면 되는 단방향 구조였기 때문에, WebSocket은 오버 엔지니어링에 가까웠음

2. “그냥 polling이면 되지 않을까?”라는 생각

- 음악 생성 시간은 평균적으로 이미지 3장 기준 약 5초 내외
- 그렇다면 프론트에서 5초 뒤에 한 번 상태를 조회하면 대부분 완료됐을 것으로 판단
- 간단한 구현, 서버 리소스 최소 사용이라는 장점이 있었음

하지만 이 판단은 초기 가정(5초 이내 작업 완료)에 의존한 것이었고,
실시간성과 UX를 완전히 보장할 수는 없었음

3. 기획 변경 및 프론트 피드백 → polling 한계 부각

- 이후 기획이 변경되며, 음악 생성 전에 이미지 기반 정보 추출 과정이 추가됨
→ 평균 생성 시간이 길어지고, polling의 실효성도 낮아짐

- 프론트엔드 측에서도 polling이 반복될 경우 불필요한 네트워크 트래픽, 서버 부하, 지속적인 setTimeout 관리 등의 문제를 제기

즉, 간단한 구현 방식이 오히려 시스템에 부담이 될 수 있음이 드러남

4. "메인화면 진입 시 상태 조회" 방식 고려

- 사용자는 이미지 업로드 후 메인화면으로 이동하여 자신의 풍등과 다른 풍등을 함께 감상
- 이 흐름을 활용해, 메인화면 진입 시점에 DB에서 음악 생성 상태(music_status)를 확인하고, 완료된 풍등만 응답에 포함시키는 방식을 검토함

이 방식은 추가 통신 채널 없이도 상태를 전달할 수 있어 구조가 단순하고 서버 부하가 없음

하지만 이는 진입 시점에서 한 번만 확인 가능하므로,

사용자가 메인화면에 머무르는 동안 상태가 바뀌는 것을 실시간으로 반영할 수 없음 → UX 저하 우려

5. 실시간 반영을 위한 대안 고민: WebSocket vs SSE

- 대기화면/전시 콘텐츠를 감상 중에도 사용자 풍등의 상태가 실시간으로 반영되어야 한다는 피드백 수용
- 다시 실시간 통신 방식 검토: WebSocket vs SSE

항목	WebSocket	SSE
통신 방향	양방향	단방향 (서버 → 클라이언트)
구현 난이도	상대적으로 복잡	간단
연결 유지 비용	높음 (ping/pong, 상태 관리 필요)	낮음 (HTTP 기반, 자동 재연결 지원)
메시지 다양성	다중 이벤트 처리 유리	단일 메시지 스트림에 적합
우리 서비스 적합도	과함	최적

6. 최종 선택: SSE

- 우리 구조는 작업 완료 상태를 단순히 푸시하면 되는 단방향 통신 구조
- 사용자 수가 많지 않아 브라우저 연결 수의 제약도 고려할 필요 없음
- SSE는 단순한 구현과 자동 재연결, 가벼운 리소스 사용량, 직관적인 흐름으로 가장 적합한 선택이었음

결과적으로 SSE는 우리 서비스의 기술 구조, 사용자 흐름, 성능 요구사항을 가장 잘 만족하는 통신 방식

6-1. SSE 사용 시 주의할 점

(1) 브라우저 연결 수 제한

- HTTP/1.1 기준, 브라우저는 도메인당 동시 연결 수(일반적으로 6개)에 제한이 있음
- SSE는 하나의 연결을 지속적으로 점유하기 때문에,
다른 요청(Fetch, Axios 등)이 많은 페이지에서는 연결 제한에 걸릴 수 있음

- 최근 브라우저 및 인프라 대부분은 HTTP/2 이상을 지원
하지만, 네트워크 환경, 프록시, CDN 설정 등에 따라 여전히 HTTP/1.1이 사용되는 경우

✅ 백엔드에서 처리한 항목

항목	분석
유저당 SSE 연결 수	이 API는 특정 lantern_id 기반으로 1개의 SSE 연결을 생성합니다 → 연결 수 최소화
메시지 구분 방식	event: music_done_partial, event: music_done_all, event: error로 명확한 이벤트 타입 구분 → 한 연결 내에서 서로 다른 메시지를 구분할 수 있도록 함
다중 이벤트 처리	한 SSE 스트림에서 여러 이벤트를 다룸 → 적절하게 단일 연결에서 다양한 이벤트 처리하고 있음 → 즉 연결을 여러 개 만들 필요가 없다

(2) 네트워크 단절 시 자동 재연결

- SSE는 기본적으로 연결이 끊기면 자동 재연결(기본 3초)을 시도함
- 하지만 서버가 재시작되거나 클라이언트가 중간 이벤트를 놓쳤을 경우 이벤트 유실 위험 존재

✅ 백엔드에서 처리한 항목

항목	분석
이벤트 ID 부여	각 이벤트(music_done_partial, music_done_all)에 id: 필드 부여 → 유실 방지를 위한 기준 제공
Last-Event-ID 수신	Header(last_event_id)로 클라이언트의 마지막 수신 ID를 읽음
resume 분기 처리	resume=true일 때만 Last-Event-ID를 사용하여 해당 ID 이후 이벤트만 전송
이벤트 유실 방지 구조	sent_task_ids 집합에 last_event_id를 포함시켜 중복 전송 방지
기본 재연결 지원	EventSourceResponse(ping=15) 설정으로 브라우저의 자동 재연결 유도

(3) 단방향 통신만 가능

- SSE는 서버 → 클라이언트 방향으로만 데이터를 전송할 수 있음
- 클라이언트 → 서버로의 데이터 전송은 별도의 HTTP 요청(fetch 등)이 필요

✅ 백엔드에서 처리한 항목

항목	분석
서버 → 클라이언트 전용	모든 메시지는 서버에서 클라이언트로 push됨
클라이언트 요청은 없음	클라이언트는 요청 없이 수신만 하므로 구조에 적합

(4) Content-Type 및 CORS 설정

- SSE는 반드시 Content-Type: text/event-stream 헤더가 포함되어야 하며, 크로스 도메인 환경에서는 CORS 설정도 필요

✅ 백엔드에서 처리한 항목

항목	설명
Content-Type: text/event-stream	EventSourceResponse가 자동으로 설정해줌 (Starlette 내부 구현)
Cache-Control: no-cache	FastAPI + Starlette 기본값으로 포함됨
Connection: keep-alive	SSE 특성상 자동 포함됨 (끊어지지 않도록 유지)
Access-Control-Allow-Origin	CORSMiddleware에 localhost:5173 등록됨
Access-Control-Allow-Credentials	allow_credentials=True로 설정됨
Nginx buffering off 설정	로컬 개발만 하고 있다면 해당 없음, 배포 시 nginx.conf 설정 필수 → 배포 진행 시 처리 할 예정